Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu

Chạy đoạn này trước khi cố giải thích nó. Nếu bạn đoán a và b khác nhau sau dòng b[0] = 99, đó là một dự đoán rất hợp lý - và cũng là cánh cửa vào phần này.

```
a := []int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a)
fmt.Println(b)
```

Kết quả từ lab part2-values là:

```
[99 20 30]
[99 20 30]
```

Đừng ghi nhớ riêng một câu “slice nguy hiểm”. Thay vào đó, ta sẽ lần ngược từ kết quả: Go đã copy cái gì khi chạy b := a, và dữ liệu nào không được copy?

Một baseline để dự đoán

Với một int, assignment tạo ra một giá trị độc lập. Cả a và b đều có giá trị riêng của chính chúng.

```
a := 10
b := a
b = 99

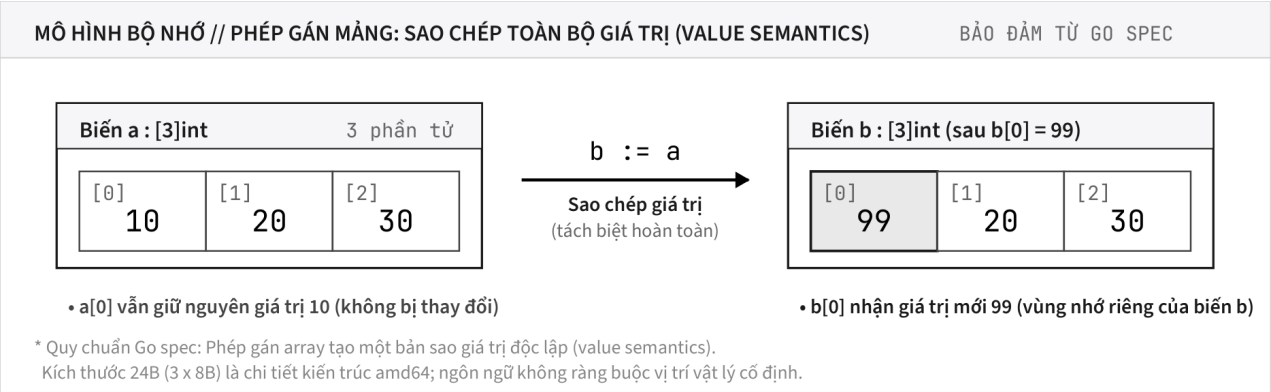
fmt.Println(a, b) // 10 99
```

Go đã copy value của a vào b. Sau đó b nhận value mới; a không có lý do gì để đổi. Đây là quy tắc nền: assignment copy value. Điều còn thiếu là một value có thể chứa gì.

Array giúp ta kiểm tra quy tắc ấy với nhiều phần tử. Độ dài là một phần của type, nên [3]int và [4]int là hai array type khác nhau. Khi array được gán, toàn bộ array value được copy.

```
a := [3]int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a) // [10 20 30]
fmt.Println(b) // [99 20 30]
```



Hình 1 — Assignment của array copy các phần tử. Việc thay b[0] chỉ chạm vào array b.

Đến đây, trực giác “copy nghĩa là độc lập” vẫn đúng. Sự thay đổi xuất hiện chỉ khi ta bỏ một ký tự trong type: [3]int trở thành []int.

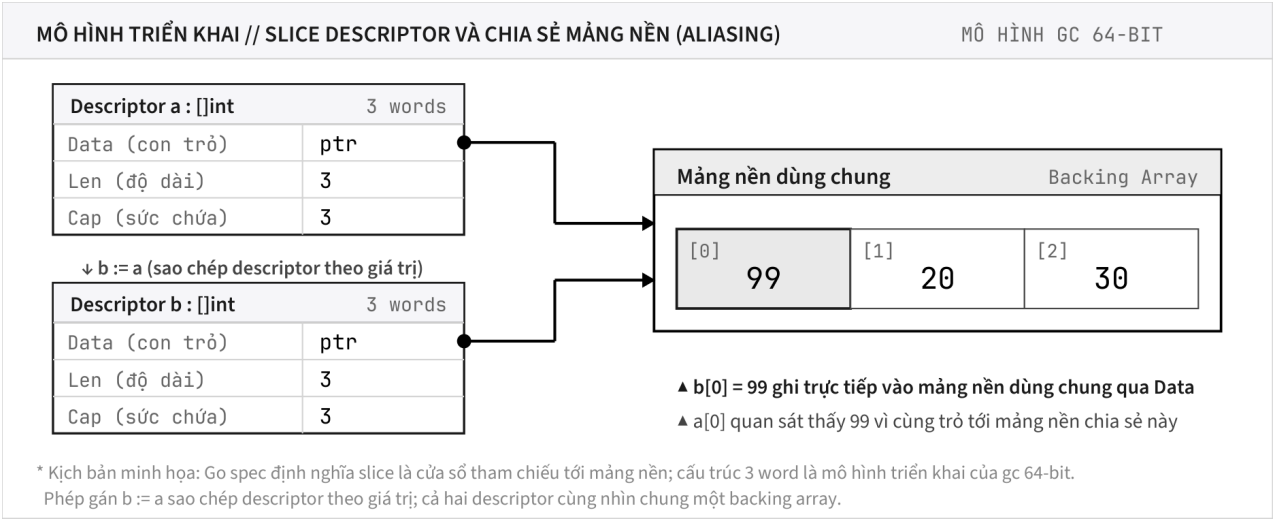
Khoảnh khắc slice đổi kết quả

```
a := []int{10, 20, 30}
b := a
b[0] = 99
```

[]int không phải array không ghi độ dài. Nó là slice type. Một slice value mô tả một đoạn liên tiếp của underlying array.

Trong source runtime Go 1.27.1 (`src/runtime/slice.go`), mô hình descriptor gồm con trỏ tới dữ liệu, `len int` và `cap int`. Với target amd64 đang dùng, ba trường này tương ứng 24 byte. Đây là biểu diễn đã pin, không phải kích thước mà specification bắt mọi implementation phải giữ. Để dự đoán mutation ở đây, chỉ cần biết slice nhìn vào array nền và có length, capacity riêng.

Khi phép gán `b := a` diễn ra, ngôn ngữ thực hiện sao chép giá trị theo ngữ nghĩa (copy by value). Trong mô hình ba từ máy, các thành phần của descriptor được sao chép sang `b` với cùng con trỏ dữ liệu, cùng `len` và cùng `cap`. Trình tối ưu hóa có thể giữ các trường này trên thanh ghi mà không bắt buộc phải phát sinh một lệnh chép 24 byte vật lý ở bộ nhớ. Biến `b` sở hữu một bản sao giá trị riêng biệt về cửa sổ nhìn (`len` và `cap`), nhưng con trỏ bên trong vẫn dẫn về cùng mảng nền ban đầu.



Hình 2 — b := a copy slice value, gồm cửa sổ nhìn vào dữ liệu, len và cap. Cả hai descriptor vẫn dẫn tới cùng underlying array, nên b[0] thay đổi phần tử mà a cũng nhìn thấy.

Vì vậy hai câu sau cùng đúng, và không mâu thuẫn:

Bảng 1 — Copy value và chia sẻ storage là hai tầng khác nhau của cùng một phép gán.

Câu nói	Đúng ở đâu?
Assignment copy value.	b nhận một slice descriptor riêng: đổi len của b bằng reslice không tự đổi len của a.
Hai slice có thể cùng đổi dữ liệu.	Slice descriptor cùng tham chiếu một backing array; mutation một element hiện ra qua mọi slice cùng nhìn element đó.

Từ aliasing dùng cho tình huống nhiều đường tham chiếu đến cùng dữ liệu. Nó không phải lỗi tự thân. Aliasing giúp cắt một cửa sổ trên buffer mà không copy cả buffer. Nó thành bug khi một người đọc tưởng mình có dữ liệu riêng nhưng thực tế lại đang sửa vùng dùng chung.

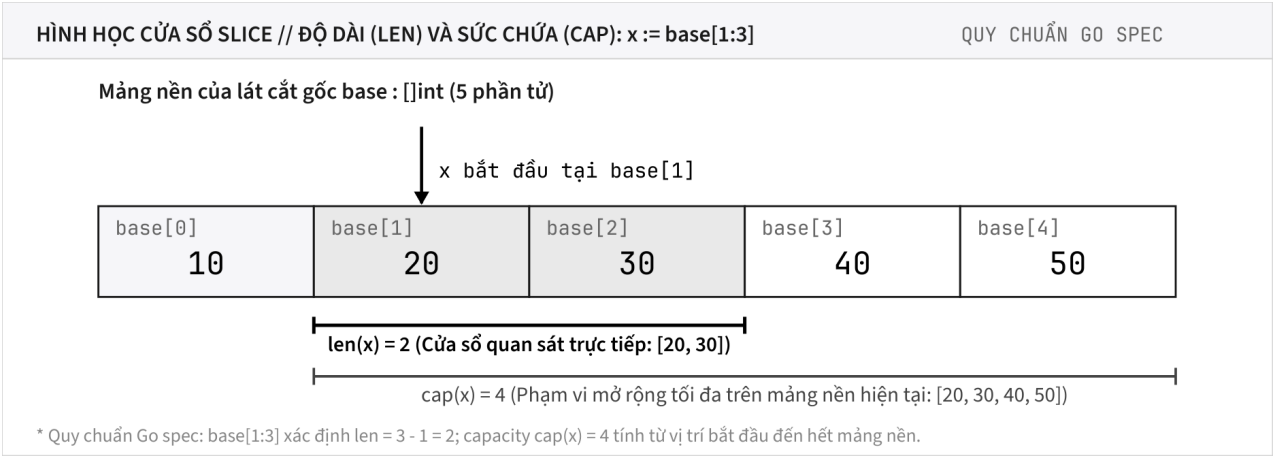
Cửa sổ, length và capacity

Đặt một slice lên một slice khác để thấy descriptor thực sự đang mô tả gì.

```
base := []int{10, 20, 30, 40, 50}
x := base[1:3]

fmt.Println(x)      // [20 30]
fmt.Println(len(x)) // 2
fmt.Println(cap(x)) // 4
```

x bắt đầu ở phần tử 20 của base và kết thúc trước 40. Vì thế length là 2. Capacity đếm số phần tử từ điểm bắt đầu của x đến cuối backing array mà x hiện có thể mở rộng tới; ở đây là 20, 30, 40, 50, nên là 4.



Hình 3 — x nhìn thấy hai phần tử đầu của cửa sổ. cap(x) không nói x đang có bốn phần tử; nó nói x có thể được reslice hoặc append trong phạm vi backing array hiện tại.

Hãy tự dự đoán trước khi chạy:

```
base := []int{1, 2, 3, 4}
a := base[:2]
b := base[1:3]
a[1] = 99

fmt.Println(base)
fmt.Println(a)
fmt.Println(b)
```

Kết quả là [1 99 3 4], [1 99], [99 3]. a[1] và b[0] đều chỉ vào phần tử base[1]. Hình vẽ không thay thế được việc lần index: nó chỉ cho biết vì sao có thể có một phần tử chung để lần.

append là một phép thử về capacity

append trả về slice kết quả. Go specification phân biệt hai trường hợp: nếu capacity hiện tại đủ cho các phần tử được thêm, append dùng lại underlying array; nếu không đủ, append cấp underlying array mới đủ lớn cho kết quả. Vì slice trả về có thể đổi độ dài hoặc mảng nền, dạng thông thường là `s = append(s, value)`. Quy tắc dùng lại mảng nền là ngữ nghĩa của ngôn ngữ; cách chọn capacity mới khi phải tăng mảng là chi tiết triển khai.

```
s := make([]int, 2, 4)
s[0], s[1] = 10, 20
old := s

s = append(s, 30)
s[0] = 99

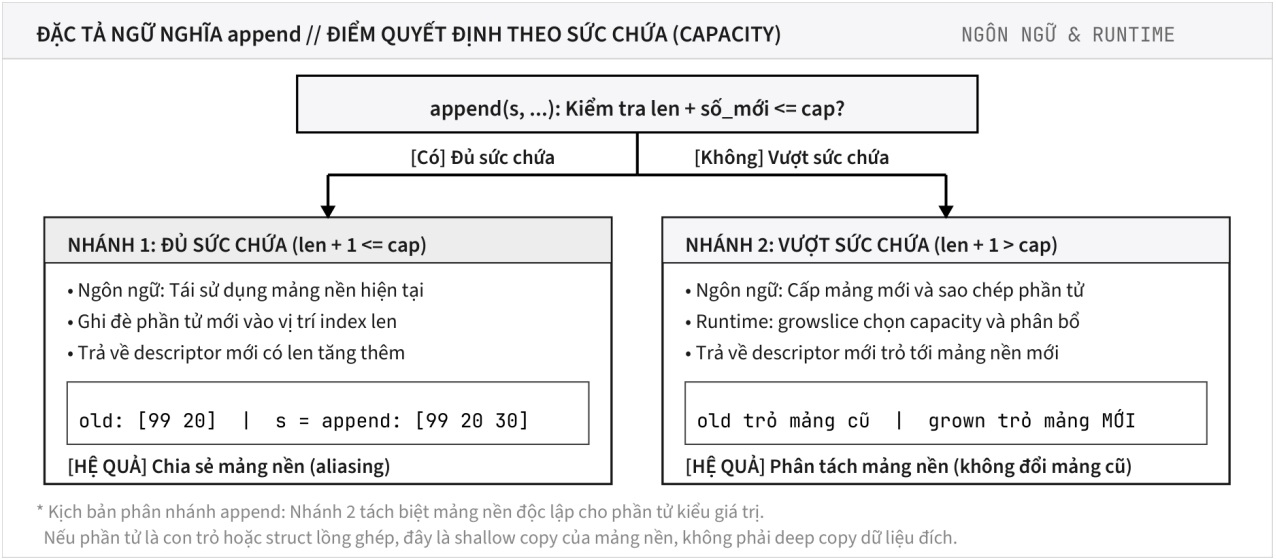
fmt.Println(old) // [99 20]
fmt.Println(s)   // [99 20 30]
```

Ở đây `s` còn capacity. `old` và `s` có length khác nhau, nhưng vẫn cùng nhìn hai element đầu của backing array. Không dùng chuỗi capacity tình cờ đo được để thiết kế ownership. Trong Go 1.27.1, `nextslicecap` ở `runtime/slice.go` chọn capacity mới theo cả length cần có và capacity cũ; nếu length mới vượt hai lần capacity cũ, nó có thể chọn thẳng length mới. Đường thông thường tăng gấp đôi dưới ngưỡng 256, rồi chuyển dần về mức tăng nhỏ hơn; `growslice` còn tính kích thước element và làm tròn allocation. Đây là thuật toán triển khai, không phải luật để dự đoán mọi lần `append` hay kết luận dữ liệu nhất thiết nằm trên heap.

Để chủ động phòng vệ, Go cung cấp cú pháp lát cắt ba chỉ số đầy đủ `s[low:high:max]` (Full Slice Expression). Cú pháp này giới hạn sức chứa của lát cắt mới ở mức `max - low`. Khi ta đặt `max = high` (như `s[:n:n]`), capacity của lát cắt mới bị giới hạn đúng bằng độ dài nhìn thấy; thao tác `append` thêm phần tử vào lát cắt này sẽ không thể dùng tiếp phần capacity nằm ngoài phạm vi quan sát của lát cắt cũ, loại bỏ nguy cơ vô tình ghi đè lên các phần tử phía sau của mảng nền chung:

```
old := []int{10, 20}
s := old[:2:2]
grown := append(s, 30)
grown[0] = 99

fmt.Println(old)   // [10 20]
fmt.Println(grown) // [99 20 30]
```



Hình 4 — Hình mô tả hai semantics mà code phải chịu được. Đừng suy luận allocation từ một capacity tình cờ; nếu code cần một kết quả riêng, hãy tạo bản copy có chủ ý.

Một aliasing bug có người dùng thật

Một function “chỉ chuẩn bị preview” đôi khi phá dữ liệu của caller.

```
func redactPreview(fields []string) {
    fields[0] = "***"
}

requestFields := []string{"api-key", "region"}
redactPreview(requestFields)
fmt.Println(requestFields) // [*** region]
```

Parameter fields nhận một slice value mới, nhưng descriptor ấy vẫn reference storage của requestFields. Việc đổi element là mutation shared storage. Nếu mục tiêu thật sự là tạo preview độc lập, copy phần tử sang một slice mới trước:

```
func redactedCopy(fields []string) []string {
    preview := make([]string, len(fields))
    copy(preview, fields)
    preview[0] = "***"
    return preview
}
```

copy sao chép tối đa số phần tử vừa với destination và trả về số phần tử đã copy. Ở đây make tạo backing array mới, nên preview độc lập ở tầng slice elements. Đổi lấy independence là allocation và chi phí copy; với buffer lớn hoặc đường xử lý nóng, đó là một quyết định cần có lý do. Với một preview cần an toàn khỏi mutation, nó diễn đạt đúng intent.

Thực hành - bug hunt có kiểm chứng. Mở `labs/part2-values` trong VS Code và chạy riêng `TestRedactedCopyDoesNotMutateInput` trước khi sửa source. Sau đó thay phần tạo `preview` bằng `preview := fields`. Đừng đoán bằng mắt: chạy lại test để thấy nó báo input đã bị mutate. Khôi phục code, xóa thân `redactedCopy`, rồi tự viết lại từ contract của test mà không nhìn implementation cũ. Đây là một bước nhỏ, nhưng nó buộc mental model "copy slice value vẫn có thể chia sẻ storage" đi qua một failure thật.

Đáp án — chỉ đọc sau khi đã tự làm. `preview := fields` chỉ copy slice descriptor, nên `preview[0]` vẫn sửa phần tử chung với `requestFields`. Bản đúng cần một backing array mới, chẳng hạn `make` rồi `copy`. Test phải giữ cả hai assertion: preview bị redacted và input không đổi; bỏ assertion thứ hai là bỏ mất contract ownership.

Một cửa sổ nhỏ giữ cả căn phòng

Độc lập khi sửa dữ liệu chưa phải câu hỏi cuối cùng về ownership. Giả sử service nhận một buffer lớn, lấy 16 byte đầu làm mã nhận diện rồi giữ mã ấy trong lịch sử. Buffer không còn cần cho nghiệp vụ. Anh dự đoán: giới hạn cả length lẫn capacity xuống 16 có cho phép GC thu hồi phần còn lại không?

```
buf := make([]byte, 4<<20)
preview := buf[:16:16]
fmt.Println(len(preview), cap(preview)) // 16 16
```

Hãy phân biệt ba câu hỏi. `len` cho biết bao nhiêu phần tử đang nằm trong cửa sổ; `cap` giới hạn cửa sổ có thể mở rộng tới đâu bằng reslice hoặc dùng lại storage khi append. Còn GC phải hỏi object nào vẫn được tham chiếu.

`preview` vẫn dẫn tới underlying array của `buf`. Đối với allocation lớn trong thí nghiệm này, còn giữ cửa sổ là còn giữ object ấy, không phải chỉ 16 byte đầu. Biểu thức ba index thay quyền mở rộng, không chuyển quyền sở hữu sang một allocation độc lập.

Ngay cả khi đặt `buf = nil`, nếu chương trình vẫn sử dụng `preview`, đường tham chiếu tới array còn tồn tại. Ngược lại, biến còn nằm trong phạm vi từ vựng không bảo đảm compiler coi nó còn sống đến cuối scope. Khi đo, ta cần giữ kết quả thực sự được dùng; lab ở Chương 10 dùng `runtime.KeepAlive` để bảo vệ ý đồ của thí nghiệm. Không suy ra số byte heap chính xác chỉ từ cú pháp `make`: vị trí cấp phát và cách tối ưu là chuyện phải quan sát trên đúng toolchain.

Anh hãy mở `labs/part10-resource-retention/baseline/retention.go`, đọc `Prefix`, nhưng chưa đọc bản sửa. Viết test với hai yêu cầu: sửa prefix không đổi input, và prefix không dùng chung byte đầu với input khi độ dài khác zero. Bản ba index sẽ thất bại ở cả hai yêu cầu dù `cap(prefix) == len(prefix)`. Test ấy kiểm chứng sharing; profile sau nhiều chu kỳ mới cho thấy hệ quả về lượng bộ nhớ giữ lại.

Nếu contract là trả về dữ liệu độc lập, lời giải trực tiếp đã nằm trong thao tác copy mà ta vừa học:

```
func Prefix(buf []byte, n int) []byte {
    out := make([]byte, n)
    copy(out, buf[:n])
    return out
}
```

Tiền điều kiện ở đây là `0 <= n && n <= len(buf)`. Với prefix 16 byte, ta giữ 16 phần tử byte trong storage riêng thay vì kéo dài tuổi thọ của toàn bộ buffer đầu vào. Đây là copy các byte, không phải lời hứa giải phóng tức thì buffer cũ: nếu một alias khác còn giữ buffer, nó vẫn sống; nếu không, GC còn cần chạy. Chi phí cấp phát cũng đổi, nên chỉ copy ở boundary thật sự cần sở hữu độc lập, không rải copy để che mọi vấn đề.

`bytes.Clone(buf[:n])` cũng phù hợp khi API làm việc với byte slice. `slices.Clone` dùng được với slice tổng quát nhưng là shallow copy: clone một `[]*Record` tạo storage mới cho các pointer, không clone những `Record` mà chúng trỏ tới. Hai hàm có contract về bản sao; không dùng capacity quan sát được của chúng làm bằng chứng về kích thước allocation. Bản `make` và `copy` trong lab cố ý đơn giản để ta kiểm tra ownership mà không phải học thêm generic API ở đây.

Một view lớn sống lâu có thể hoàn toàn hợp lệ nếu caller vẫn cần toàn bộ dữ liệu. Trường hợp đáng sửa là tuổi thọ thực tế vượt nhu cầu của API: nghiệp vụ chỉ giữ một mã nhỏ nhưng implementation vô tình giữ cả buffer. Ta gọi đó là giữ bộ nhớ ngoài ý muốn, **memory retention**. GC không bị hỏng; contract về dữ liệu cần sống bao lâu đang bị implementation làm rộng hơn.

String: bytes trước, text sau

String cũng là value, nhưng nó có luật riêng: string là immutable sequence of bytes. len đếm byte; index trả về byte. Text thường là UTF-8, song Go không yêu cầu mọi string phải chứa UTF-8 hợp lệ.

```
text := "Việt"
fmt.Println(len(text)) // 6

for index, value := range text {
    fmt.Println(index, value)
}
```

Lab chạy đoạn này với kết quả:

```
6
0 86
1 105
2 7879
5 116
```

V và i mỗi chiếm một byte. ệ được mã hóa bằng ba byte UTF-8, vì vậy rune tiếp theo bắt đầu ở byte index 5. range trên string decode UTF-8: giá trị thứ

hai là rune, thường dùng để biểu diễn Unicode code point. byte là alias của uint8; rune là alias của int32. Code point vẫn không đồng nghĩa với "một ký tự người dùng nhìn thấy": một grapheme có thể gồm nhiều code point. Ta chưa cần giải quyết segmentation ở đây; chỉ cần không nhầm len với số ký tự hiển thị.

String không cho phép gán trực tiếp `text[0] = 'v'`. Với thao tác theo byte trong ví dụ này, chuyển sang `[]byte`, sửa slice rồi tạo string kết quả là một cách rõ ràng. Đó không phải con đường duy nhất tạo text mới: thao tác theo điểm mã có thể dùng rune, còn ghép chuỗi có thể dùng builder. Chọn đơn vị dữ liệu trước khi chọn công cụ sửa nó.

Conversion giữa string và byte slice phải giữ semantics của giá trị: sửa byte slice tạo từ string không được làm đổi string gốc; string tạo từ slice giữ giá trị dù slice về sau bị sửa. Specification không bắt một lần cấp phát hay physical copy ở mọi biểu thức. Compiler có thể bỏ allocation/copy khi chứng minh behavior quan sát không đổi. Nếu cần biết một biểu thức có allocate không, đo và xem output compiler của đúng version thay vì suy ra từ syntax.

Pointer là câu hỏi khác

Sau slice, pointer dễ bị giải thích sai như "slice nhưng rõ hơn". Pointer là một value trực tiếp reference một variable khác; slice là một descriptor cho segment của underlying array. Chúng có thể cùng dẫn tới sharing, nhưng chúng trả lời hai câu hỏi khác nhau.

```
count := 10
p := &count
*p = 99
fmt.Println(count) // 99
```

Ở đây &count tạo pointer tới variable count, còn *p truy cập variable đó. Pointer sẽ có chương riêng khi ta cần thiết kế API quanh mutation trực tiếp và nil. Trước khi đến đó, hãy giữ model vừa xây: luôn hỏi value nào được copy, và value ấy có reference tới storage hay variable nào không.

Nếu câu trả lời là “không”, assignment thường tạo independence như int và array. Nếu câu trả lời là “có”, aliasing có thể là công cụ tốt hoặc một bug - tùy việc bạn có làm rõ ownership và mutation hay không.

Một quyết định ở ranh giới API

Khi một hàm nhận hoặc trả về slice, câu hỏi thiết kế không phải là “Go có copy không?” mà là: **ai được phép sửa storage này, và trong khoảng thời gian nào?** Một hàm lọc dữ liệu có thể trả về một view để tránh allocation; khi đó caller cần biết view ấy còn dùng chung buffer. Một hàm dựng response hoặc preview thường nên trả về dữ liệu độc lập, vì caller không có lý do để đoán một thay đổi sau đó sẽ đi ngược vào input.

Không có đáp án mặc định miễn phí. Copy tốn allocation và thời gian; chia sẻ tiết kiệm chúng nhưng làm contract về ownership quan trọng hơn. Trước khi tối ưu, hãy viết test cho hành vi mutation mà API hứa. Sau đó, khi đọc một bug khó hiểu, lần theo ba bước: value nào được gán, value đó reference storage nào, và ai còn có thể chạm storage ấy. Đó là cách biến “slice lạ quá” thành một cuộc điều tra có thể kiểm chứng.

1. Go Team. Go Specification, Slice types và Full slice expressions. go.dev/ref/spec#Slice_types; go.dev/ref/spec#Full_slice_expressions
2. Go Team. Package `bytes`, `Clone`, Go 1.27.1. pkg.go.dev/bytes@go1.27.1#Clone
3. Go Team. Package `slices`, `Clone`, Go 1.27.1. pkg.go.dev/slices@go1.27.1#Clone
4. Go Team. Go Garbage Collector Guide, Tracing Garbage Collection. go.dev/doc/gc-guide#Tracing_Garbage_Collection